WIKIPEDIA
25 years of the free encyclopedia

k-way merge algorithm

In computer science, ***k*-way merge algorithms** or multiway merges are a specific type of sequence merge algorithms that specialize in taking in k sorted lists and merging them into a single sorted list. These merge algorithms generally refer to merge algorithms that take in a number of sorted lists greater than two. Two-way merges are also referred to as binary merges. The k-way merge is also an external sorting algorithm.

Two-way merge

A 2-way merge, or a binary merge, has been studied extensively due to its key role in merge sort. An example of such is the classic merge that appears frequently in merge sort examples. The classic merge outputs the data item with the lowest key at each step; given some sorted lists, it produces a sorted list containing all the elements in any of the input lists, and it does so in time proportional to the sum of the lengths of the input lists.

Denote by A[1..p] and B[1..q] two arrays sorted in increasing order. Further, denote by C[1..n] the output array. The canonical 2-way merge algorithm[1] stores indices i, j, and k into A, B, and C respectively. Initially, these indices refer to the first element, i.e., are 1. If A[i] < B[j], then the algorithm copies A[i] into C[k] and increases i and k. Otherwise, the algorithm copies B[j] into C[k] and increases j and k. A special case arises if either i or j have reached the end of A or B. In this case the algorithm copies the remaining elements of B or A into C and terminates.

k-way merge

The *k*-way merge problem consists of merging k sorted arrays to produce a single sorted array with the same elements. Denote by n the total number of elements. n is equal to the size of the output array and the sum of the sizes of the k input arrays. For simplicity, we assume that none of the input arrays is empty. As a consequence $k \leq n$, which simplifies the reported running times. The problem can be solved in $\mathcal{O}(n \cdot \log(k))$ running time with $\mathcal{O}(n)$ space. Several algorithms that achieve this running time exist.

Iterative 2-way merge

The problem can be solved by iteratively merging two of the k arrays using a 2-way merge until only a single array is left. If the arrays are merged in arbitrary order, then the resulting running time is only O(kn). This is suboptimal.

The running time can be improved by iteratively merging the first with the second, the third with the fourth, and so on. As the number of arrays is halved in each iteration, there are only Θ(log k) iterations. In each iteration every element is moved exactly once. The running time per iteration is therefore in Θ(n) as n is the number of elements. The total running time is therefore in Θ(n log k).

We can further improve upon this algorithm, by iteratively merging the two shortest arrays. It is clear that this minimizes the running time and can therefore not be worse than the strategy described in the previous paragraph. The running time is therefore in O(n log k). Fortunately, in border cases the running time can be better. Consider for example the degenerate case, where all but one array contain only one element. The strategy explained in the previous paragraph needs Θ(n log k) running time, while the improved one only needs Θ(n + k log k) running time.

Direct *k*-way merge

In this case, we would simultaneously merge k-runs together.

A straightforward implementation would scan all k arrays to determine the minimum. This straightforward implementation results in a running time of Θ(kn). Note that this is mentioned only as a possibility, for the sake of discussion. Although it would work, it is not efficient.

We can improve upon this by computing the smallest element faster. By using either heaps, tournament trees, or splay trees, the smallest element can be determined in O(log k) time. The resulting running times are therefore in O(n log k).

The heap is more commonly used, although a tournament tree is faster in practice. A heap uses approximately 2*log(k) comparisons in each step because it handles the tree from the root down to the bottom and needs to compare both children of each node. Meanwhile, a tournament tree only needs log(k) comparisons because it starts on the bottom of the tree and works up to the root, only making a single comparison in each layer. The tournament tree should therefore be the preferred implementation.

Heap

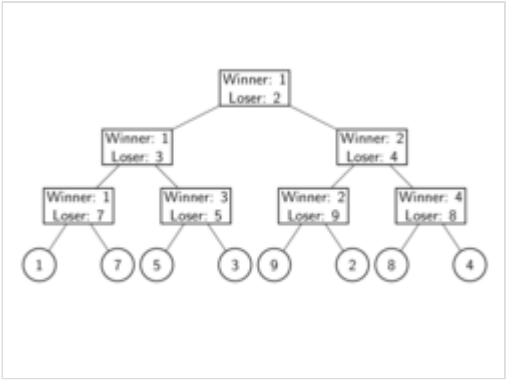
The idea is to maintain a min-heap of the k lists, each keyed by their smallest current element. A simple algorithm builds an output buffer with nodes from the heap. Start by building a min-heap of nodes, where each node consists of a head element of the list, and the rest (or tail) of the list. Because the lists are sorted initially, the head is the smallest element of each list; the heap property guarantees that the root contains the minimum element over all lists. Extract the root node from the heap, add the head element to the output buffer, create a new node out of the tail, and insert it into the heap. Repeat until there is only one node left in the heap, at which point just append that remaining list (head and tail) to the output buffer.

Using pointers, an in-place heap algorithm [2] allocates a min-heap of pointers into the input arrays. Initially these pointers point to the smallest elements of the input array. The pointers are sorted by the value that they point to. In an O(k) preprocessing step the heap is created using the standard heapify procedure. Afterwards, the algorithm iteratively transfers the element that the root pointer points to, increases this pointer and executes the standard increase key procedure upon the root element. The running time of the increase key procedure is bounded by O(log k). As there are n elements, the total running time is O(n log k).

Note that the operation of replacing the key and iteratively doing increase-key or sift-down are not supported by many Priority Queue libraries such as C++ stl and Java. Doing an extract-min and insert function is less efficient.

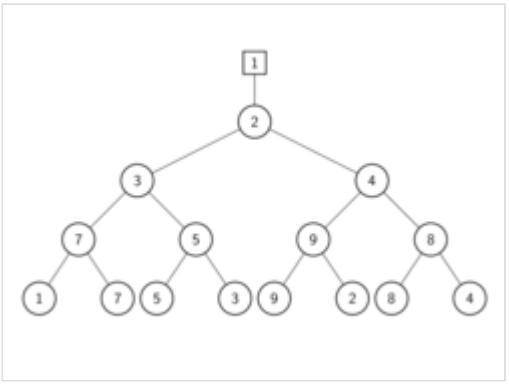
Tournament Tree

The Tournament Tree [3] is based on an elimination tournament, as in sports competitions. In each game, two of the input elements compete. The winner is promoted to the next round. Therefore, we get a binary tree of games. The list is sorted in ascending order, so the winner of a game is the smaller one of both elements.



Tournament tree

For k-way merging, it is more efficient to only store the loser of each game (see image). The data structure is therefore called a loser tree. When building the tree or replacing an element with the next one from its list, we still promote the winner of the game to the top. The tree is filled like in a sports match but the nodes only store the loser. Usually, an additional node above the root is added that represents the overall winner. Every leaf stores a pointer to one of the input arrays. Every inner node stores a value and an index. The index of an inner node indicates which input array the value comes from. The value contains a copy of the first element of the corresponding input array.



Loser tree

The algorithm iteratively appends the minimum element to the result and then removes the element from the corresponding input list. It updates the nodes on the path from the updated leaf to the root (*replacement selection*). The removed element is the overall winner. Therefore, it has won each game on the path from the input array to the root. When selecting a new element from the input array, the element needs to compete against the previous losers on the path to the root. When using a loser tree, the partner for replaying the games is already stored in the nodes. The loser of each replayed game is written to the node and the winner is iteratively promoted to the top. When the root is reached, the new overall winner was found and can be used in the next round of merging.

The images of the tournament tree and the loser tree in this section use the same data and can be compared to understand the way a loser tree works.

Algorithm

A tournament tree can be represented as a balanced binary tree by adding sentinels to the input lists (i.e. adding a member to the end of each list with a value of infinity) and by adding null lists (comprising only a sentinel) until the number of lists is a power of two. The balanced tree can be stored in a single array. The parent element can be reached by dividing the current index by two.

When one of the leaves is updated, all games from the leaf to the root are replayed. In the following pseudocode, an object oriented tree is used instead of an array because it is easier to understand. Additionally, the number of lists to merge is assumed to be a power of two.

```
function merge(L1, ..., Ln)
    buildTree(heads of L1, ..., Ln)
    while tree has elements
```

```
        winner := tree.winner
        output winner.value
        new := winner.index.next
        replayGames(winner.parent, new) // Replacement selection
```

```
function replayGames(node, new)
    loser, winner := playGame(node, new)
    node.value := loser.value
    node.index := loser.index
    if node != root
        replayGames(node.parent, winner)
```

```
function buildTree(elements)
    nextLayer := new Array()
    while elements not empty
        el1 := elements.take()
        el2 := elements.take()
        loser, winner := playGame(el1, el2)
        parent := new Node(el1, el2, loser)
        nextLayer.add(parent)
    if nextLayer.size == 1
        return nextLayer // only root
    else
        return buildTree(nextLayer)
```

Running time

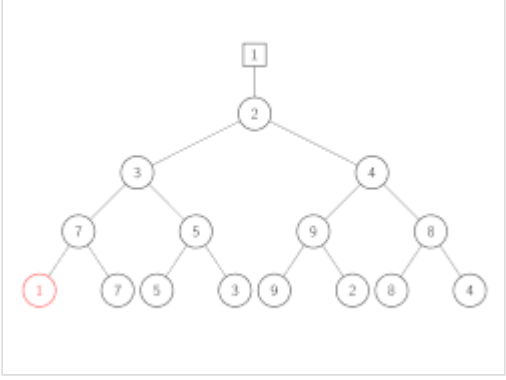
In the beginning, the tree is first created in time Θ(k). In each step of merging, only the games on the path from the new element to the root need to be replayed. In each layer, only one comparison is needed. As the tree is balanced, the path from one of the input arrays to the root contains only Θ(log k) elements. In total, there are n elements that need to be transferred. The resulting total running time is therefore in Θ(n log k).[3]

Example

The following section contains a detailed example for the replacement selection step and one example for a complete merge containing multiple replacement selections.

Replacement selection

Games are replayed from the bottom to the top. In each layer of the tree, the currently stored element of the node and the element that was provided from the layer below compete. The winner is promoted to the top until we found the new overall winner. The loser is stored in the node of the tree.



Example for replacement selection

Step	Action
1	Leaf 1 (overall winner) is replaced by 9, the next element from the input list.
2	Replaying the game 9 vs 7 (previous loser). 7 wins because it is smaller. Therefore, 7 is promoted to the top while 9 is saved in the node.
3	Replaying the game 7 vs 3 (previous loser). 3 wins because it is smaller. Therefore, 3 is promoted to the top while 7 is saved in the node.
4	Replaying the game 3 vs 2 (previous loser). 2 wins because it is smaller. Therefore, 2 is promoted to the top while 3 is saved in the node.
5	The new overall winner 2 is saved above the root.

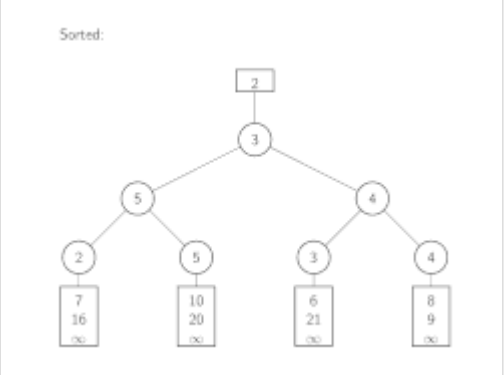
Merge

To execute the merge itself, the overall smallest element is repeatedly replaced with the next input element. After that, the games to the top are replayed.

This example uses four sorted arrays as input.

```
{2, 7, 16}
{5, 10, 20}
{3, 6, 21}
{4, 8, 9}
```

The algorithm is initiated with the heads of each input list. Using these elements, a binary tree of losers is built. For merging, the lowest list element 2 is determined by looking at the overall minimum element at the top of the tree. That value is then popped off, and its leaf is refilled with 7, the next value in the input list. The games on the way to the top are replayed like in the previous section about replacement selection. The next element that is removed is 3. Starting from the next value in the list, 6, the games are replayed up until the root. This is being repeated until the minimum of the tree equals infinity.



Visualization for the whole algorithm

Lower bound on running time

One can show that no comparison-based *k*-way merge algorithm exists with a running time in $O(n \text{ f(k)})$ where f grows asymptotically slower than a logarithm, and n being the total number of elements. (Excluding data with desirable distributions such as disjoint ranges.) The proof is a straightforward reduction from comparison-based sorting. Suppose that such an algorithm existed, then we could construct a comparison-based sorting algorithm with running time $O(n \text{ f}(n))$ as follows: Chop the input

array into n arrays of size 1. Merge these n arrays with the *k*-way merge algorithm. The resulting array is sorted and the algorithm has a running time in $O(n\ f(n))$. This is a contradiction to the well-known result that no comparison-based sorting algorithm with a worst case running time below $O(n \log n)$ exists.

External sorting

k-way merges are used in external sorting procedures.[4] External sorting algorithms are a class of sorting algorithms that can handle massive amounts of data. External sorting is required when the data being sorted do not fit into the main memory of a computing device (usually RAM) and instead they must reside in the slower external memory (usually a hard drive). *k*-way merge algorithms usually take place in the second stage of external sorting algorithms, much like they do for merge sort.

A multiway merge allows for the files outside of memory to be merged in fewer passes than in a binary merge. If there are 6 runs that need be merged then a binary merge would need to take 3 merge passes, as opposed to a 6-way merge's single merge pass. This reduction of merge passes is especially important considering the large amount of information that is usually being sorted in the first place, allowing for greater speed-ups while also reducing the amount of accesses to slower storage.

References

1. Thomas H. Cormen; Charles E. Leiserson; Ronald L. Rivest; Clifford Stein (2001). *Introduction To Algorithms* (https://books.google.com/books?id=NLngYyWFl_YC&pg=PA11). MIT Press. pp. 28–29. ISBN 978-0-262-03293-3.
2. Bentley, Jon Louis (2000). *Programming Pearls* (2nd ed.). Addison Wesley. pp. 147–162. ISBN 0201657880.
3. Knuth, Donald (1998). "Chapter 5.4.1. Multiway Merging and Replacement Selection". *Sorting and Searching*. The Art of Computer Programming. Vol. 3 (2nd ed.). Addison-Wesley. pp. 252–255. ISBN 0-201-89685-0.
4. Shaffer, Clifford A. (2012-07-26). *Data Structures and Algorithm Analysis in C++, Third Edition* (https://books.google.com/books?id=AijEAgAAQBAJ). Courier Corporation. ISBN 9780486172620.

Retrieved from "https://en.wikipedia.org/w/index.php?title=K-way_merge_algorithm&oldid=1358267528"